

Project Overview

This project implements a real-time, interactive video and image filtering application that combines classical image processing with advanced depth-based neural network techniques. The primary goal is to provide an experimental platform for exploring visual effects, pattern recognition and computer vision concepts in an intuitive and interactive way, with most filters implemented from scratch rather than using OpenCV's pre-existing functions.

Built using **C++**, **OpenCV** and **ONNX Runtime**, the application supports static image display, live video capture and GPU-accelerated depth estimation. Core features include filters such as greyscale, sepia, Gaussian blur, Sobel edge detection, embossing, negative and color quantization, as well as creative effects like selective color isolation, vignette, exposure adjustment and face detection.

Integration of **Depth Anything V2** in the application enables per-pixel depth estimation in real time that drives innovative effects including bokeh (portrait-style depth-of-field), depth-based color grading and multiple colormap visualizations.

The software architecture emphasizes modularity and reuse via a shared filterApp module, allowing both image and video pipelines to leverage identical filter logic, where stackable filters provide flexibility for dynamic and interactive experimentation.

Required Tasks

Task 1: Static Image Display (imgDisplay.cpp)

Implemented a program to read an image file and display it in a window. The program enters an event loop checking for keypresses, with 'q' to quit, 's' to save screenshots and other key presses for applying additional filter and/or effects to the image.

Task 2: Live Video Capture (vidDisplay.cpp)

Created a video capture program that opens the camera (video channel) and enters a loop to display live video in a window and handles keyboard input with 'q' for quitting, 's' for saving screenshots and other key presses for applying additional filter and/or effects to live video feed/frame.

Task 3: OpenCV Greyscale Conversion

Implemented greyscale video display using OpenCV's `cvtColor` function, which gets triggered when the user presses 'g' key. Grayscale images and videos are a type of image representation where each pixel contains only the intensity (brightness) of light, without

any color information. In a grayscale video, all the color information is removed, leaving only varying shades of gray. This makes grayscale videos more efficient in terms of storage and processing.

When converting to grayscale, each RGB channel is *not* weighted uniformly. Instead, we weight each channel differently to account for how much color we perceive of each:

$$\text{Grey} = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

These weights reflect human eye sensitivity, where green contributes most to perceived brightness, followed by red, then blue.

Required Image 1: Original and OpenCV Greyscale Comparison



Original Image



OpenCV gray-scale image

Task 4: Custom Greyscale Implementation

Implemented a custom greyscale conversion using the minimum channel method, which gets triggered when the user presses 'h' key:

$$\text{Grey} = \min(R, G, B)$$

I chose the minimum channel value method because it emphasizes the darkest component of each pixel, producing higher contrast and a more dramatic visual effect compared to OpenCV's built-in grayscale conversion.

Additionally, while OpenCV's `cvtColor` converts the image to a single-channel intensity map, this approach retains all three-color channels, setting each to the minimum value. This preserves a 3-channel structure, allowing subsequent color-based filters to be applied directly without converting back.

This approach balances brightness across channels, resulting in a smoother, more natural grayscale. This causes shadows and darker areas to appear more pronounced, highlights are less bright and the overall image has a stronger contrast. This method can be useful for artistic effects or when emphasizing edges and darker regions in a scene.

Required Image 2: Custom Greyscale with Differences Highlighted



OpenCV gray-scale image



Custom gray-scale image [min(R,G,B)]

Task 5: Sepia Tone Filter

Implemented sepia tone filter using matrix transformation to create an antique photograph effect, which gets triggered when the user presses 'p' key. The transformation applies to all three color channels simultaneously:

$$\text{New Blue} = 0.272 \times R + 0.534 \times G + 0.131 \times B$$

$$\text{New Green} = 0.349 \times R + 0.686 \times G + 0.168 \times B$$

$$\text{New Red} = 0.393 \times R + 0.769 \times G + 0.189 \times B$$

To ensure correct calculation, the original RGB values for each pixel were stored in temporary variables before computing the new channel values. This prevents intermediate updates from affecting subsequent calculations. After applying the transformation, the new value for each channel is clipped to the range [0, 255] to avoid overflow, since the sum of coefficients can exceed 1.0. This approach preserves the intended sepia effect while maintaining valid pixel intensity values across all channels.

Required Image 3: Sepia Tone Filter Result



Sepia-tone filter

Task 6: 5×5 Gaussian Blur Filter

Implementation 1: Naive Approach (blur5x5_1)

Implemented 5×5 Gaussian blur using direct 2D convolution with the `at<>` accessor method that uses the following integer approximated Gaussian kernel:

```
[1 2 4 2 1]
[2 4 8 4 2]
[4 8 16 8 4]
[2 4 8 4 2]
[1 2 4 2 1]
```

Implementation 2: Optimized Separable Filters (`blur5x5_2`)

Optimized implementation of 5×5 Gaussian blur using separable 1×5 filters: horizontal `[1 2 4 2 1]` followed by vertical `[1 2 4 2 1]T`. Uses `ptr<>` method for direct memory access instead of `at<>` bounds checking. This effect can be used by pressing the 'b' key.

Changes

Two changes were made to make second implementation of Blur filter faster than the original implementation.

1. First change was the use of separable filters.

The original implementation required 25 multiplications per pixel, 24 additions to calculate the final value and finally 1 normalization step which required dividing by 100 to bring the pixel values back to the original scale.

This makes it a total of 50 operations to process a given pixel.

In the second implementation that uses two separable filters to achieve the same result, the number of multiplications required were 5, the number of additions required were 4 and then there was the final normalization step.

This makes it a total of 10 operations to process a given pixel.

And because this happened twice because there are two filters in this case and both need to iterate over the image to produce the desired result the total number of operations required would become 20 per pixel.

From these calculations we can see that there was a significant reduction in the number of steps that had to be carried out to calculate the value of a pixel from original implementation to the faster implementation.

2. Second change was the use of `ptr<>` approach.

In our original approach we were asked to use the `at<>` approach. Now, OpenCV stores images in interleaved format. With the `at<>` approach we have to access every individual cell using indexing which adds a computational overhead as for each pixel, the program has to find the given row, access the given row to get to the required column, then find the required column, get the value from there and finally multiply it with the weight to compute the final multiplication value.

In the faster implementation I used `ptr<>` approach. In `ptr<>` approach we add pointers to for all the rows that will be used in the computation of the given pixel. In this case, a maximum of 5 rows would be used in the computation of final

multiplication value, in case of vertical separable filter, so we add pointers to all the 5 rows that will be used to calculate the weighted sum for a given pixel. This way we don't have to find the row as we already have a pointer pointing to that row. All the program needs to do is, in the given row, get to the required column, get the pixel value from there and finally multiply it with the weight to compute the final multiplication value. (This can also be referred to as caching of rows in the memory which ultimately results in a shorter turnaround time.)

Using the above 2 techniques in the faster implementation results in a significant reduction in the operation time required to process a given pixel thereby reducing the overall time taken by the program and making the implementation faster.

Performance Comparison

Time per image (1): 0.2234 seconds (**Naïve Blur**)

Time per image (2): 0.0294 seconds (**Optimized Blur**)

Required Image 4: Original and Blurred Image



Original Image



Blurred Image

Task 7: Sobel Edge Detection Filters

Implemented 3×3 Sobel X and Y filters for edge detection. Both filters use 16-bit signed output (CV_16SC3) to handle negative gradient values in the range [-255, 255]. In the live video feed, Sobel X filter gets triggered when the user presses 'x' key and Sobel Y filter gets triggered when the user presses 'y' key.

Sobel X Filter (Horizontal Gradient)

Detects vertical edges with positive response to the right. Kernel:

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Sobel Y Filter (Vertical Gradient)

Detects horizontal edges with positive response upward. Kernel:

```
[ 1  2  1]
[ 0  0  0]
[-1 -2 -1]
```

Task 8: Gradient Magnitude

Implemented gradient magnitude computation using Euclidean distance, which gets triggered when the user presses 'm' key:

$$\text{magnitude} = \sqrt{(sx^2 + sy^2)}$$

Combines both Sobel X and Y to detect all edges regardless of orientation. Output is converted to 8-bit unsigned for display.

Required Image 5: Sobel and Gradient Magnitude Results



Original Image



Sobel X



Sobel Y



Gradient Magnitude

Task 9: Blur and Quantize (Cartoon Effect)

Implemented a cartoon-style filter combining Gaussian blur with color quantization, which uses 10 levels per channel and gets triggered when the user presses 'l' key.

$$\text{Bucket size } b = 255 / 10 = 25$$

For each pixel value x : $\text{bucket} = x / b$, $\text{quantized} = \text{bucket} \times b$

Result: $10^3 = 1,000$ possible colors (reduced from 16.7 million), creating distinct color bands and cartoon-like appearance.

Required Image 6: Blur Quantize Effect



Original Image



Blur-Quantized Image

Task 10: Haar Cascade Face Detection

Integrated Haar cascade face detection using the provided faceDetect.cpp code. The system detects faces in real-time and draws bounding boxes around detected faces. This effect gets triggered when the user presses 'f' key.

Required Image 7: Face Detection in Video

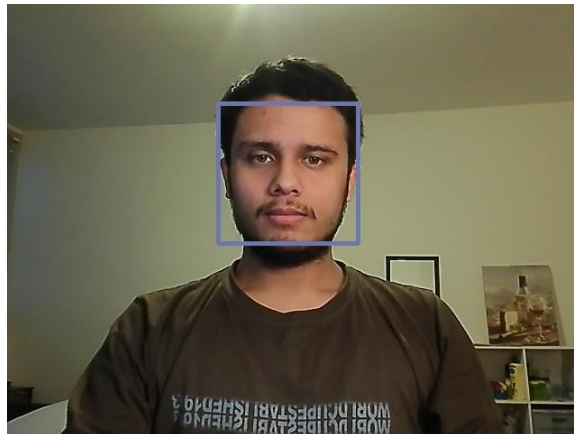


Image with Face Detection Enabled

Task 11: Depth Estimation (Depth Anything V2)

Integrated Depth Anything V2 neural network using ONNX Runtime for real-time depth estimation. The network provides per-pixel depth estimates enabling sophisticated depth-based filtering effects. To use this filter the user needs to press the 'd' key. By pressing the trigger key repeatedly, the user can cycle through the depth modes. (I configured my program to use the GPU version of ONNX runtime. I switched to the GPU version once I completed all the requirements of this assignment using CPU version.)

Depth Visualization Modes

Implemented multiple depth visualization colormaps:

- Hot Mode (INFERNO colormap): Blue=far, Red/Orange=close
- Cool Mode (COOL colormap): Cyan=far, Purple=close
- Bokeh Effect (Depth-based portrait mode)
- Color graded depth mode (Depth-based coloring mode)

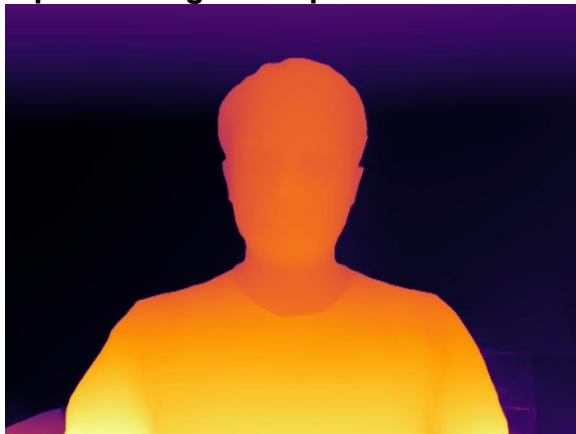
Creative Depth-Based Filter: Bokeh Effect

Developed a depth-based bokeh (portrait mode) effect that simulates professional camera depth-of-field:

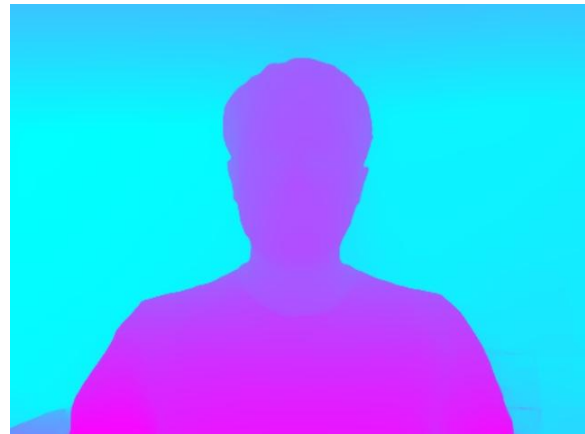
- Objects close to camera (depth > 180): Sharp and in focus
- Objects far from camera (depth < 180): Progressively blurred
- Blend factor: depth / threshold, creates smooth transition

The depth-based bokeh effect is implemented by first generating a per-pixel depth map using the Depth Anything V2 network. A heavily blurred version of the original frame is created using a large Gaussian kernel (21×21). For each pixel, a blend factor is computed based on the depth value: pixels with depth greater than the focus threshold (180) remain sharp, while closer pixels are progressively blended with the blurred image using a linear interpolation (blend = depth / threshold). The final output is a smooth combination of the original sharp image and the blurred background, producing a portrait-style depth-of-field effect where the subject remains in focus while the background gradually blurs.

Required Image 8: Depth Visualization



Depth-Visualization: Hot Mode



Depth-Visualization: Cool Mode

Required Image 9: Creative Depth Filter (Bokeh Effect)



Original Image



Depth-Visualization: Bokeh Effect

Task 12: Additional Effects

Effect 1: Emboss Effect (3D Relief) - Area Computation

Implemented embossing effect using Sobel X and Y gradients. Computes dot product with light direction vector (0.7071, 0.7071) to create 3D relief appearance. This filter can be applied by pressing the 'z' key.

$$\text{Emboss} = (0.7071 \times sx + 0.7071 \times sy) + 128$$

The 128 offset centers the result at mid-gray, creating a raised surface effect.

Required Image 10: Emboss Effect



Original Image



Embossed Image

Effect 2: Negative (Color Inversion) - Pixel-wise

Implemented color inversion (negative) effect by inverting each color channel. This filter can be used by pressing the 'n' key.

$$\text{newValue} = 255 - \text{originalValue}$$

Creates film negative effect where colors become their complements on the color wheel (red→cyan, blue→yellow, etc.).

Required Image 11: Negative Effect



Original Image



Negative Image

Effect 3: Depth-Based Color Grading - Depth-Based

Implemented cinematic color grading based on depth information:

- Close objects (depth > 170): Warm red/orange tones
- Mid-distance (85-170): Natural colors preserved
- Far objects (depth < 85): Cool blue/cyan tones

This depth-based color grading enhances perceived depth by leveraging human sensitivity to color temperature. Warmer colors such as reds and oranges tend to appear closer to the viewer, while cooler colors like blues and cyans visually recede. Using the per-pixel depth map, close objects are gradually shifted toward warmer tones, far objects toward cooler tones and mid-range regions remain unchanged. The strength of the color shift is proportional to the depth distance, creating a smooth transition rather than hard boundaries. This results in a cinematic depth cue that reinforces spatial separation in the scene without altering geometry, making depth relationships more intuitive and visually compelling.

Required Image 12: Depth Color Grading Effect



Original Image



Depth-Visualization: Color Grading

Extensions

Beyond the required tasks, I implemented several extensions to explore advanced filtering techniques and software architecture patterns.

Extension 1: Shared Filter Application Architecture

Designed and implemented a shared filterApp.cpp module that abstracts all filter logic and UI handling. This allows both imgDisplay and vidDisplay to use identical filter functionality without code duplication.

Key architectural benefits:

- DRY Principle: Filter logic written once along with key press integration, used by both image and video processing programs.
- Lambda functions: Clean abstraction for different input sources (static image vs video stream).
- Maintainability: Updates to filters automatically apply to both modes
- Professional design: Separation of concerns between input handling and processing

Extension 2: Additional Creative Filters/Effects

1. Vignette – Effect

Implemented radial gradient darkening towards image edges for cinematic appearance, which can be enabled by pressing the 'v' key. Uses distance from center with quadratic falloff and adjustable intensity (0.0-2.0).

The user can use '+' key for increasing and '-' key for decreasing the intensity of vignette effect.

The plus point of this is that as an effect this can be applied to any of the primary underlying filters and it provides the same level of intensity control on all the filters wherever visually possible.



Vignette on Original Image



Vignette on Sepia Filter



Vignette on Emboss Filter



Vignette on Negative Filter

2. Temperature Control (Warmth/Coolness) – Effect

Implemented independent temperature control effects, that can be triggered using 'w' key for warmth effect and 'e' key for coolness effect. Similar to vignette effect these can be applied on any of the primary filters. An added functionality of these features is that these are cyclic in nature. Once the intensity of these effects reaches to their max values they reset to their base value (If I press 'w' key when warmth is at an intensity of 2.0 then it will reset to 0.0)

- Warmth: Boosts red/orange tones, reduces blue (sunset/golden hour effect)
- Coolness: Boosts blue/cyan tones, reduces red (winter/moonlight effect)

Both use cycling intensity (0.0-2.0) and can be combined for creative color grading.



Warmth on Original Image



Warmth on Sepia Filter



Coolness on Emboss Filter



Coolness on Negative Filter

3. Exposure Control – Effect

Another effect that I implemented is the multiplicative exposure adjustment (brightness control) simulating camera exposure settings. Range: 0.5× (darker) to 1.5× (brighter), controlled with '1' and '2' keys ('1' for increasing the exposure and '2' for decreasing the exposure).



Decreased Exposure on Original Image



Decreased Exposure on Sepia Filter



Increased Exposure on Emboss Filter



Increased Exposure on Negative Filter

4. Selective Color Isolation – Filter

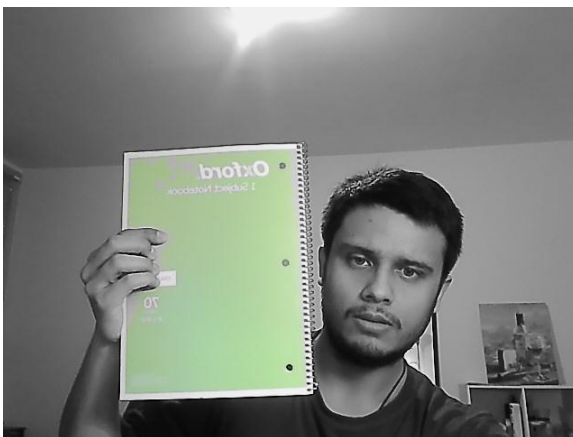
Implemented color isolation filter that preserves pixels of a target color while converting everything else to greyscale. Uses HSV color space for accurate hue-based selection. Supports red, blue, green and yellow isolation modes, creating dramatic selective color effects common in artistic photography. This effect is a cyclic effect like depth analysis modes and can be triggered using 'o' key.



Red Color Detection Mode



Blue Color Detection Mode



Green Color Detection Mode



Yellow Color Detection Mode

Extension 3: Video Recording with Filters

Implemented video recording capability that captures filtered video output. This feature can be enabled and disabled manually using 'k' key. Additional features include:

- 5-second countdown before recording starts.
- Visual REC indicator on the top-left of display window (not recorded in output).
- Real-time progress updates in seconds.
- MP4 format with H.264 codec.
- Maximum 30 seconds (900 frames) with auto-stop.
- Filter changes during recording supported.
- Camera mode detection (only available in live video feed mode).

Refer the below link for a demo of video recording capability

<https://northeastern.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=f1d7799f-e3cf-453e-bb0a-b3de001418f1>

Extension 4: Multiple Depth Visualization and Processing Modes

Expanded depth functionality beyond basic visualization with four distinct modes accessible via cycling:

- Hot Visualization: INFERNO colormap (red/orange theme)
- Cool Visualization: COOL colormap (blue/cyan theme)
- Bokeh Effect: Depth-based selective blur (portrait mode)
- Color Grading: Depth-based temperature adjustment

Extension 5: Additional Features and Effects

Implemented several additional filters and effects:

- Mirror/Flip: Horizontal flip effect ('r' key press)
- Sparkle Halo: Face detection-based decorative effect ('t' key press)
- Independent Intensity Controls: Separate adjustments for vignette, warmth, coolness and exposure
- Filter Pipeline: Modular architecture allowing stacking of multiple effects

Reflection

This project significantly deepened my understanding of image processing and computer vision by requiring hands-on implementation of core filtering techniques rather than relying on built-in library functions. Implementing filters such as Gaussian blur, Sobel edge detection and embossing clarified how convolution kernels operate at the pixel level and how separable filters can drastically improve performance. Working in C++ with OpenCV for real-time image and video processing strengthened my understanding of memory access patterns, data types and performance considerations required for interactive applications.

Integrating Depth Anything V2 provided valuable insight into modern depth estimation using neural networks and how depth information can be combined with classical image processing to create advanced effects like bokeh and depth-based color grading. One of the main challenges was configuring ONNX Runtime, particularly switching to the GPU version and ensuring compatibility with CUDA, which required careful environment and build management. What surprised me most was how small architectural choices—such as shared filter pipelines and pointer-based access—had a large impact on performance and maintainability. Overall, this project improved my intuition for image structure, spatial relationships and real-time processing and the skills gained will be directly applicable to future work in computer vision and graphics-oriented systems.

Acknowledgements

This assignment was completed with reference to a variety of academic, technical and online resources. I took conceptual assistance from large language models (Claude and ChatGPT) to understand depth-based image processing concepts, particularly how depth maps can be used to drive visual effects such as bokeh and smooth edge transitions, as well as for guidance on configuring ONNX Runtime with GPU support, setting up the CMake build system and for paraphrasing and checking the grammatical correctness of this document.

Additional resources consulted include:

- OpenCV documentation and tutorials (<https://docs.opencv.org>, <https://opencv.org>)
- ONNX Runtime documentation and examples (<https://onnx.ai>)
- Depth Anything V2 project page and documentation (<https://depth-anything-v2.github.io>)
- Hugging Face model repository for depth estimation models (<https://huggingface.co>)
- NVIDIA Developer documentation for CUDA and cuDNN installation and configuration (<https://developer.nvidia.com>)
- - Microsoft Visual Studio Build Tools documentation for C++ compilation <https://learn.microsoft.com/en-us/cpp/>
- CMake documentation for build system configuration <https://cmake.org/documentation/>
- GitHub repositories and issue trackers for ONNX Runtime troubleshooting
- OpenCV GitHub discussions for filter implementation examples <https://github.com/opencv/opencv>
- Professor Maxwell's DA2Network wrapper class, face detection and time blur code files and example materials
- Course lecture notes and assignment handouts
- Piazza class discussion board for troubleshooting ONNX Runtime and Windows compatibility issues

Specific online references used for filter design and color processing:

- Stack Overflow: Creating a vignette filter in OpenCV <https://stackoverflow.com/questions/22654770/creating-vignette-filter-in-opencv>
- Stack Overflow: HSV color ranges reference <https://stackoverflow.com/questions/12357732/hsv-color-ranges-table>
- GeeksforGeeks: Vignette filter implementation <https://www.geeksforgeeks.org/python/create-a-vignette-filter-using-python-opencv/>
- General reference searches and articles via Google and Wikipedia for other filters and color theory

Appendix: Complete Feature List

The final application includes 25+ interactive features organized into three categories:

Primary Filters (Mutually Exclusive)

- OpenCV Greyscale ('g')
- Custom Greyscale ('h') - min channel method
- Sepia Tone ('p')
- Sobel X ('x') - vertical edge detection
- Sobel Y ('y') - horizontal edge detection
- Gradient Magnitude ('m')
- Blur Quantize ('l') - cartoon effect
- Negative ('n') - color inversion
- Emboss ('z') - 3D relief effect
- Depth Visualization Hot ('d' cycling)
- Depth Visualization Cool ('d' cycling)
- Depth Bokeh ('d' cycling)

- Depth Color Grading ('d' cycling)
- Color Isolation Red ('a' cycling)
- Color Isolation Blue ('a' cycling)
- Color Isolation Green ('a' cycling)
- Color Isolation Yellow ('a' cycling)

Secondary Effects (Stackable)

- Vignette ('v') - adjustable with +/- (0.0-2.0)
- 5×5 Gaussian Blur ('b')
- Warmth ('w') - cycles 0.0-2.0
- Coolness ('e') - cycles 0.0-2.0
- Face Detection ('f')
- Sparkle Halo ('t') - above detected faces
- Mirror/Flip ('r')
- Brightness Increase ('1')
- Brightness Decrease ('2')

Utility Functions

- Clear All Filters and Effects ('c')
- Display Info About Active Filters and Effects on the Live Feed ('i')
- Save Screenshot ('s')
- Record Video ('k') - camera mode only
- Quit ('q')